



KI-ENNA – Ein Neuronales Netz zum Ausprobieren

(1) Ziele von KI-ENNA

Viele KI-Tools vermitteln ein Verständnis davon, wie man KI benutzen kann. KI-ENNA zeigt, wie KI funktioniert. Dabei **trainieren Schülerinnen und Schüler interaktiv neuronale Netzwerke**, erleben die **Bedeutung von Datenqualität** und können mögliche Fehler im Training oder bei der Anwendung von **KI transparent einsehen**, um diese Schritt für Schritt zu verbessern und somit ihre **eigene KI zu erschaffen**. Dafür ermöglicht KI-ENNA die **Klassifikation zum Unterscheiden von Objekten**, deren Erkennung mit einer Kamera (im Englischen: **Computer Vision**) oder das **Training von kleinen Sprachmodellen mit einem echten Generative Transformer**, vollkommen **datenschutzkonform**, ohne spezielle Hardwarevoraussetzungen oder Vorkenntnisse der Schülerinnen und Schüler. Einfach loslegen und eine eigene KI entwickeln und ausprobieren.

(2) Didaktische Leitsätze

Viele KI-Tools sind Black Boxes. KI-ENNA ist eine Glass Box.

Schülerinnen und Schüler **trainieren selbst neuronale Netzwerke** statt nur mit KI zu chatten oder zu prompten. Dabei bilden sie **den gesamten Prozess** ab, der eine KI möglich macht: Daten sammeln und labeln sowie Modelle trainieren und testen. Darum sind mit KI-ENNA Einblicke möglich, **warum KI Fehler machen kann**, warum die **Datenqualität entscheidend** ist und dass in der Regel Menschen hinter KI-Tools stecken.

Die KI kann nicht alles wissen!

KI-ENNA zeigt: KI weiß nur das, **was man ihr zeigt**. Schlechte Daten führen zu schlechten KI-Antworten. Damit lernen Schülerinnen und Schüler **statistisches Denken** und einen **kritischen Umgang mit KI**.

KI-ENNA basiert auf Learning by Doing.

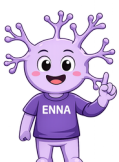
Schülerinnen und Schüler können mit KI-ENNA einer KI **interaktiv und niedrigschwellig** den Unterschied zwischen Alltagsgegenständen beibringen, das neuronale Netzwerk geometrische Formen mit einer Kamera sehen lassen oder sogar **eigene kleine Sprachmodelle trainieren**. Dadurch werden Neuronen, Aktivierungspfade und sogar die Funktionsweise eines Generative Transformers transparent einsehbar. **KI wird dabei nicht nur erklärt - sie wird erlebt.**

Offene und anpassbare Lösungen ermöglichen Kreativität und Zielgruppenspezifität.

KI-ENNA ist **Open Source**, als **MIT-lizenzierte Software** frei anpassbar und bleibt dabei **immer kostenlos**. Schülerinnen und Schüler sowie Lehrkräfte können somit **eigene Ideen einbringen**, neue Beispiele und Anwendungsmöglichkeiten entwickeln und immer **100% lokal, datenschutzkonform und unabhängig** sein. Und weil KI-ENNA keine Cloud-Anbindung oder teure Hardware voraussetzt, ist dieser didaktische Einstieg in die Welt des KI-Trainings auch noch besonders **ressourcenschonend**.

(3) Kurzbeschreibung von KI-ENNA

Für Schülerinnen und Schüler bedeutet KI-ENNA einen **Perspektivwechsel**: Weg von der reinen Nutzung digitaler Werkzeuge hin zu einem **tiefgreifenden Verständnis ihrer Grundlagen**. Dadurch werden Lernende nicht nur auf eine von KI geprägte Lebens- und Arbeitswelt vorbereitet, sondern auch in ihrer Mündigkeit gestärkt.



Klassifikation zum Unterscheiden von Objekten

(0) Objekte auswählen

Zu Beginn legt man fest, welche zwei Objekte KI-ENNA zu unterscheiden lernen soll. Dazu hinterlegt man **eigene Bilder der Objekte** in die beiden vorgesehenen Felder für **Objekt (0)** und **Objekt (1)**, oder wählt eines der **vorgegebenen Beispiele** aus. Nachfolgend ist das Beispiel der Nussfrüchte dargestellt, an dem ersichtlich wird, dass Kastanien mit (0) ein anderes Objekt darstellen, als Eicheln mit (1).

Beispiel (Noppensteine)Beispiel (Nudelgerichte)Beispiel (Nussfrüchte)

Objekt (0)



Bild hier ablegen oder klicken

Objekt (1)



Bild hier ablegen oder klicken

Bilder zurücksetzen

(1) Daten

Als nächstes sind die Objekte anhand **ausgewählter Eigenschaften** (im Englischen: **Features**) zu beschreiben. Eigenschaften können die **Beschaffenheit der Objekte** betreffen (bspw. 0.0 für eckig und 1.0 für rund), deren **Abmessungen** (bspw. 4.5 cm in der Höhe und 4.0 cm in der Breite) oder die **Farbe** (bspw. 0.0 für dunkel und 1.0 für hell).

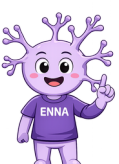
Eigene Daten

Tipp: Trage hier zeilenweise die vier Features des jeweils gezeigten Objekts ein.

	1. Feature	2. Feature	3. Feature	4. Feature
	F1	F2	F3	F4
	F1	F2	F3	F4
	F1	F2	F3	F4
	F1	F2	F3	F4

Daten übernehmenTabelle leeren

Wichtig ist, dass die ausgewählten Eigenschaften **immer in Form von Zahlen eingegeben** werden und das dazugehörige **Dezimaltrennzeichen ein Punkt** ist. Dies gilt in der internationalen Programmierung als der Standard.



(2) Neuronales Netzwerk

Modell auswählen

In den Basiseinstellungen kann zwischen einem **kleinen neuronalen Netzwerk** und einem **größeren neuronalen Netzwerk** als Modell gewählt werden. Wenn dem neuronalen Netzwerk vier Eigenschaften (im Englischen: **Features**) vorgegeben werden, befinden sich vier sogenannte **Neuronen** links in der **Eingabeschicht**. Über die Anzahl an Neuronen in den nachfolgenden Schichten, den sogenannten verborgenen Schichten, werden diese Eigenschaften bis zur **Ausgabeschicht** verarbeitet. Das einzelne Neuron in der Ausgabeschicht trifft daraufhin eine Entscheidung, bspw. zwischen Objekt (0) und Objekt (1). Die Entscheidung eines jeden Neurons basiert auf **mathematischen Berechnungen**.

Modell auswählen

Kleines Netzwerk Größeres Netzwerk

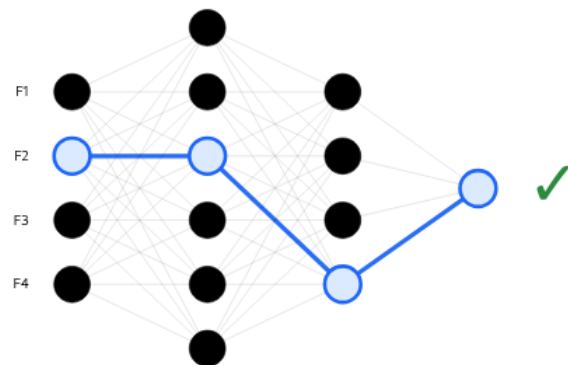
Epochen

wenige (10) oder viele (500)

Lernrate

langsam (0.001) oder schnell (1)

Training starten



Epochen

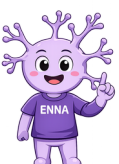
Bei einem neuronalen Netzwerk bedeutet eine **Epoche**, dass das neuronale Netzwerk einmal **alle Trainingsbeispiele komplett durchlernt**. Danach schaut das neuronale Netzwerk, welche Aufgaben es gut gelöst hat und wo es noch Fehler gemacht hat, und versucht beim nächsten Durchgang besser zu werden. Mehrere Epochen helfen dem neuronalen Netzwerk also, **aus Fehlern zu lernen und immer genauer zu werden**.

Lernrate

Die **Lernrate** sagt bei einem neuronalen Netzwerk, **wie große Schritte beim Lernen gemacht** werden sollen. Wenn das neuronale Netzwerk Fehler entdeckt, kann es diesen **nur ein kleines bisschen korrigieren** oder **gleich sehr stark ändern**. Die Lernrate bestimmt also, ob das neuronale Netzwerk **langsam in kleinen Schritten** oder **schneller in größeren Schritten** aus seinen Fehlern lernt. Eine gute Lernrate hilft dem neuronalen Netzwerk, **ruhig und Schritt für Schritt** immer besser zu werden.

Aktivierungspfad

Später, wenn das neuronale Netzwerk trainiert ist und ein **Objekt klassifiziert** werden soll, wird in dem neuronalen Netzwerk zusätzlich der sogenannte Aktivierungspfad angezeigt. Dieser verdeutlicht, wie die Eigenschaften der Objekte zur **Entscheidungsfindung des neuronalen Netzwerks** beigetragen haben und **welche Eigenschaft besonders wichtig** war.



(3) Auswertung

Die Konfusionsmatrix (im Englischen: **Confusion Matrix**) zeigt übersichtlich, wie gut ein neuronales Netzwerk als Klassifikationsmodell seine Vorhersagen trifft. Sie vergleicht die **tatsächlichen Objektklassen** mit den **vorhergesagten Objektklassen** und prüft dabei bspw., ob eine vorhergesagte Kastanie (0) auch tatsächlich eine Kastanie ist, oder eine Eichel (1). Wenn bspw. zwei Kastanien (0) für das Training des neuronalen Netzwerks verwendet worden sind, sollten auch zwei **True Negatives (TN)** mit dem Wert (0) ausgewiesen werden. Bei den Eicheln (1) wären es entsprechend **True Positives (TP)** mit dem Wert (1).

```
Konfusionsmatrix:
TN: 2  FP: 0
FN: 0  TP: 2

Genauigkeit:
1.00

🎉 Super! Das Netzwerk hat gut gelernt!
```

Hält das neuronale Netzwerk eine Eichel (1) für eine Kastanie (0), ergibt sich daraus ein **False Negative (FN)** und hält das neuronale Netzwerk eine Kastanie (0) für eine Eichel (1), so ergibt sich daraus ein **False Positive (FP)**. Wenn ein neuronales Netzwerk zu vielen False Negatives und False Positives führt, dann sinkt dessen **Genauigkeit**, welche in **Prozent angegeben** wird.

(4) Vorhersage

Im letzten Schritt kann das trainierte neuronale Netzwerk zur **Klassifikation von Objekten** verwendet werden. Andere Kastanien (0) und Eicheln (1), die nicht für das Training verwendet worden sind, können nun erkannt werden.

1. Feature

2. Feature

3. Feature

4. Feature

Objekt erkennen

Ergebnis:



Ergebnis der Vorhersage:

Vorhersage: **Objekt (0)**

Warum hat sich das neuronale Netzwerk so entschieden?

Sicherheit: **99.2 %** · Stärkster lokaler Einfluss: **Feature 2**

Feature 1

→ Objekt 0

Feature 2 ★

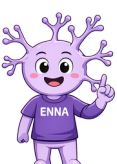
→ Objekt 0

Feature 3

→ Objekt 1

Feature 4

→ Objekt 0



Dazu trägt man wieder die Beschaffenheit der Objekte, deren Abmessungen oder Farbe als ausgewählte **Eigenschaften in die Eingabemaske** ein und das neuronale Netzwerk ermittelt den **dazugehörigen Aktivierungspfad** (siehe unter 2) sowie die **Stärke des Einflusses der Eigenschaften** und die dazugehörige **Sicherheit in der Vorhersage**. In der KI-ENNA Variante **Computer Vision** erfolgt stattdessen der **Zugriff auf eine Kamera**, um nach dem gleichen Prinzip trainierte **geometrische Formen** und die Genauigkeit in deren Erkennung direkt ausprobieren zu können.

Training von Sprachmodellen mit einem Generative Transformer

(1) Voreinstellungen

Embedding-Dimensionen

Die **Embedding-Dimensionen** beschreiben, **wie viele Zahlen** das Modell benutzt, um die **Eigenschaften eines Wortes** abzubilden. Der Transformer verarbeitet die Embeddings **kontextabhängig** und verbessert sie während des Trainings.

Embedding-Dimensionen

 14
wenige (2) oder viele (24) Dimensionen

(2) Daten und (3) Training

KI-ENNA schlägt für das Training von Sprachmodellen mit einem Generative Transformer **eigenständig geeignete Einstellungen** vor. Um die **Funktionsweise besser nachvollziehen** zu können, empfiehlt es sich, diese Einstellungen eigenständig zu variieren und deren Einfluss auf das Lernverhalten des neuronalen Netzwerks zu beobachten.

Vokabulargröße

Die Vokabulargröße beschreibt, **wie viele verschiedene Wörter oder Wortteile ein Sprachmodell kennt**. Man kann sich das wie ein Wörterbuch vorstellen: Je größer das Wörterbuch ist, desto mehr Wörter kann man verstehen und benutzen.

Beispielsätze als Trainingsdaten

Die Königin sieht den Hund.
Die Königin ruft den Hund.
Die Königin füttert den Hund.
Die Königin lobt den Hund.
Der Hund sieht die Königin.
Der Hund hört die Königin.
Der Hund folgt der Königin.
Der Hund bewacht das Schloss.
Der Hund sucht den Knochen.

Tokenisieren

Beispieldaten

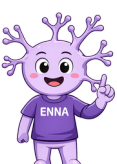
Vorgeschlagene Trainings-Hyperparameter

Vokabulargröße
28

Batches
111

Kontextlänge
9

Attention-Heads
2



Batches

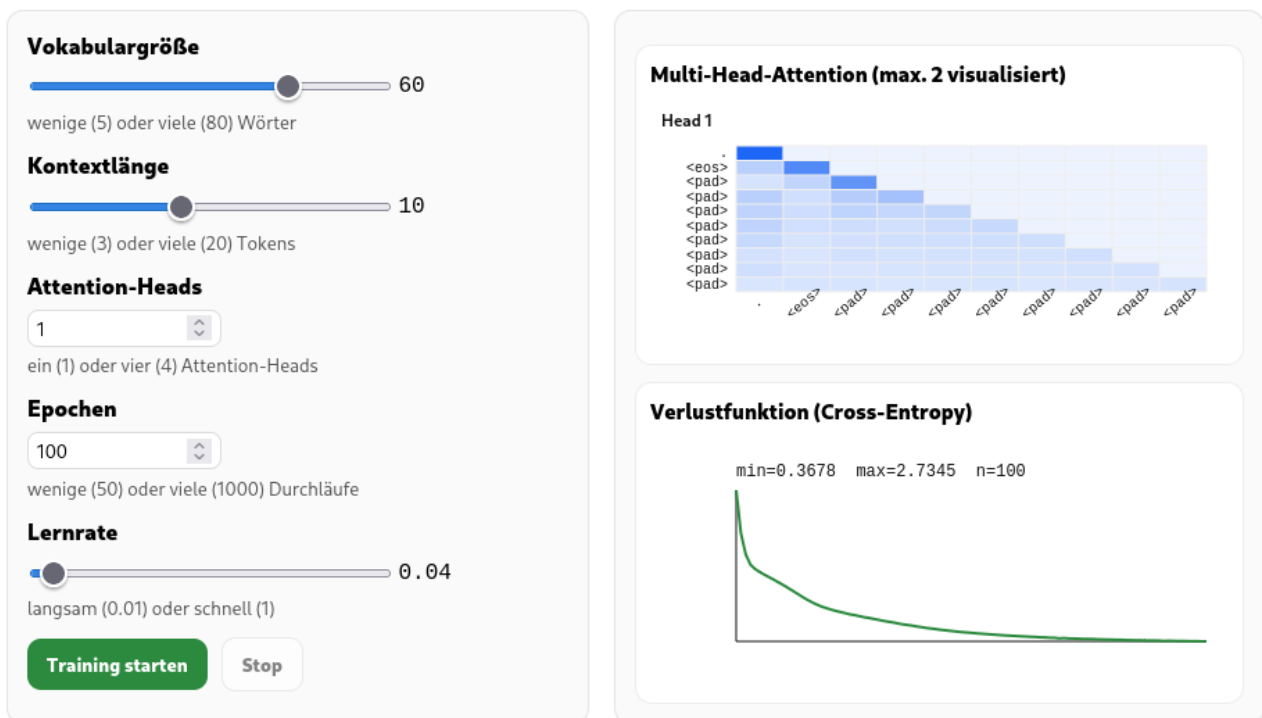
Ein Batch ist eine **kleine Gruppe von Trainingsbeispielen**, die das neuronale Netzwerk **gleichzeitig verarbeitet**. Anstatt alle Beispielsätze als Trainingsdaten auf einmal zu betrachten, **lernt das neuronale Netzwerk Schritt für Schritt** aus mehreren kleinen Gruppen. Nach jedem Batch passt es seine Einstellungen an, sodass es beim nächsten Batch bessere Vorhersagen treffen kann.

Kontextlänge

Die Kontextlänge sagt, **wie viele Wörter sich das Modell gleichzeitig merken und beachten kann**, wenn es einen Text liest oder neuen Text schreibt. Wenn sich das neuronale Netzwerk mit Transformer viele Wörter aus dem Satz davor merken kann, versteht es den Text besser.

Attention-Heads

Attention-Heads sind verschiedene **Blickrichtungen eines neuronalen Netzwerks**. Jeder Attention-Head achtet beim Verarbeiten einer Eingabe auf andere **wichtige Zusammenhänge**, zum Beispiel auf die **Bedeutung einzelner Wörter oder deren Beziehung zueinander**. So kann das neuronale Netzwerk Informationen aus unterschiedlichen Perspektiven betrachten und Zusammenhänge besser verstehen.

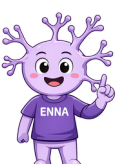


Multi-Head-Attention (max. 2 visualisiert) als Aufmerksamkeitsmatrix

Eine **visualisierte Aufmerksamkeitsmatrix** zeigt, **auf welche Wörter ein Sprachmodell beim Lesen eines Satzes besonders achtet**, sodass man sehen kann, welche Wörter für das Verständnis anderer Wörter wichtig sind.

Verlustfunktion (Cross Entropy)

Die **Verlustfunktion** ist eine **Zahl**, die misst, **wie viele Fehler das Modell noch macht**, damit das neuronale Netzwerk mit Transformer weiß, wie stark es seine Einstellungen ändern muss, um beim nächsten Lernen besser zu werden.



(4) Embeddings

Vektorisierte Wörter (Token-Vektor)

Ein neuronales Netzwerk kann Wörter nicht direkt verstehen, sondern wandelt jedes **Wort oder Token zunächst in eine Liste von Zahlen** um. Diese Zahlenliste nennt man Token-Vektor. Ähnliche Wörter besitzen häufig ähnliche Token-Vektoren, sodass das Modell **Beziehungen zwischen Wörtern** erkennen kann. Während des Trainings werden diese Zahlen schrittweise angepasst, damit die Token-Vektoren die Bedeutung und den Gebrauch der Wörter immer besser widerspiegeln. Die **Embedding-Dimensionen** legen fest, wie viele Zahlen einen Token-Vektor repräsentieren.

Token-Vektoren-Matrix

Token	Vektor
.	[-1.810, 0.255, -0.481, 2.064, 0.147, -1.507, ...]
hund	[1.361, 0.449, 0.075, 0.174, 0.549, 0.996, ...]
der	[-0.427, 0.540, 1.144, -0.183, -1.797, 0.724, ...]
königin	[1.262, -0.083, -0.919, -0.923, 0.612, 0.043, ...]
die	[-0.848, -0.307, 0.428, -0.077, -1.203, -0.861, ...]
den	[0.570, 0.254, 1.650, 0.800, -0.853, -0.091, ...]
gärtner	[0.145, 1.566, -1.004, -0.544, -0.227, -0.024, ...]
im	[-1.073, 0.314, -0.785, 0.795, 0.034, 1.228, ...]
knochen	[0.951, -0.498, -0.241, 0.907, 1.171, -0.648, ...]
sieht	[-0.445, -0.328, -0.788, -0.139, 0.183, -0.205, ...]
das	[1.451, 0.095, -0.447, 0.391, 0.047, 0.680, ...]

Token-Vektor-Matrix (Embedding-Matrix)

Die Token-Vektor-Matrix ist eine große Tabelle, in der für jedes bekannte Token ein eigener **Token-Vektor gespeichert** ist. Immer wenn das neuronale Netzwerk ein **Token verarbeitet**, liest es den passenden Vektor aus dieser Matrix aus. Während des Trainings werden die Werte der gesamten Token-Vektor-Matrix **kontinuierlich angepasst**. Dadurch lernt das Modell nach und nach bessere Darstellungen der Wörter und kann deren **Bedeutung sowie ihre Beziehungen zu anderen Wörtern** genauer erfassen.

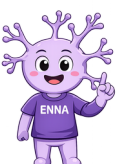
(5) Satzgenerator

Maximale Satzlänge

Mit der maximalen Satzlänge kann festgelegt werden, **wie viele Tokens für die Generierung eines Satzes verwendet werden dürfen**. Ohne eine feste Grenze würde ein Sprachmodell unter Umständen immer weiter Tokens verarbeiten.

Temperature

Die Temperature legt fest, **wie zufällig das neuronale Netzwerk das nächste Token auswählt**. Bei einer **niedrigen Temperature** entscheidet sich das Modell meist für das wahrscheinlichste Token und erzeugt dadurch eher **vorhersehbare und konsistente Texte**. Bei einer **höheren Temperature** berücksichtigt es auch weniger wahrscheinliche Token häufiger. Dadurch werden die Antworten **kreativer und abwechslungsreicher**.



Maximale Satzlänge

10

Sicherheitsgrenze (falls Satzende fehlt)

Top-k (Anzahl)

5

wenige (1) oder viele (10) Alternativen

Satz generieren

Temperatur

0.8

niedrige (0.2) oder hohe (2.0) Kreativität

Top-p (Wahrscheinlichkeit)

0.90

geringe (0.1) oder hohe (1.0) Wahrscheinlichkeit

Ergebnis

Generierter Satz:

die königin sitzt im garten.

Sampling-Parameter:

top_k=5, top_p=0.90, Temperatur=0.8

Schrittweise Wortwahl mit Generierungszustand und Alternativen:

1. Token: sitzt

Alternativen:

gibt (0.240)

füttert (0.208)

sitzt (0.192)

lobt (0.189)

sieht (0.171)

2. Token: im

Alternativen:

im (0.810)

königin (0.102)

der (0.035)

den (0.028)

die (0.025)

3. Token: garten

Alternativen:

garten (0.689)

schloss (0.311)

4. Token: .

Alternativen:

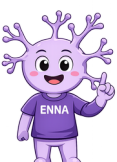
.(1.000)

Top-k

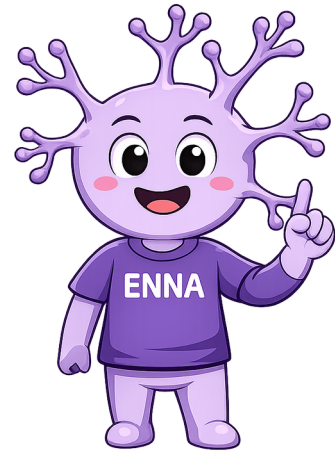
Top-k begrenzt die Auswahl der möglichen nächsten Token. Das neuronale Netzwerk betrachtet also nur diese **begrenzte Auswahl** und wählt daraus das nächste Token aus. Ein **kleiner Top-k-Wert führt zu eher sicheren und vorhersehbaren Antworten**, während ein größerer Wert mehr Vielfalt und kreativere Formulierungen ermöglicht.

Top-p (Nucleus Sampling)

Top-p begrenzt die Auswahl der möglichen nächsten Token auf die **wahrscheinlichsten Token**, deren **gemeinsame Wahrscheinlichkeit einen bestimmten Anteil** erreicht. Das neuronale Netzwerk berücksichtigt also nur so viele Token, wie nötig sind, um diesen Wahrscheinlichkeitsanteil abzudecken. Dadurch passt sich die Größe der Auswahl automatisch an die jeweilige Situation an und erzeugt häufig einen guten Ausgleich zwischen zuverlässigen und abwechslungsreichen Antworten.



SO LERNT



ICH BRAUCHE BEISPIELE

Du zeigst mir Dinge und ihre Eigenschaften.
Diese Eigenschaften nennt man Features.
Je mehr gute Beispiele Du mir gibst,
desto besser kann ich lernen.

Deshalb ist eine
sorgfältige Vorauswahl
der Eigenschaften
besonders wichtig!

ICH SUCHE NACH MUSTERN

Ich vergleiche alle Beispiele miteinander.
Dabei suche ich nach Gemeinsamkeiten
und Unterschieden. So erkenne ich,
welche Eigenschaften besonders wichtig sind.

ICH LERNE MIT VERSCHIEDENEN BEISPIELEN

Statt alle Beispiele auf einmal anzusehen,
lerne ich mit einer Auswahl verschiedener
Beispiele. Eine solche Auswahl nennt man Batch.
Dadurch kann ich schneller lernen.

Um die Gemeinsamkeiten und
Unterschiede zu entdecken,
sind möglichst viele
Beispiele erforderlich!

Ein neuronales Netz kann zu wenige
oder zu viele Epochen haben und es
kann zu schnell oder zu langsam lernen!

ICH LERNE IMMER WIEDER

Eine Lernrunde nennt man Epoche.
In jeder Epoche schaue ich mir alle Beispiele
noch einmal an. Mit jeder Epoche werde
ich meistens ein bisschen besser.

ICH LERNE AUS MEINEN FEHLERN

Am Anfang liege ich oft daneben.
Nach jeder Lernrunde überprüfe ich meine
Vorhersagen. War ich falsch, ändere ich meine
Berechnungen ein kleines Stück.

In den Neuronen werden die
gewichteten Eingangssignale durch
Aktivierungsfunktionen verarbeitet,
wodurch schrittweise eine Vorhersage
entsteht, die mit den Beispielen
verglichen wird.

Dies ermöglicht die erforderliche
Flexibilität, um ungelernte Dinge zu
erkennen, birgt aber auch das Risiko
einer Fehlerkennung!

ICH MERKE MIR KEINE FESTEN REGELN

Ich lerne keine festen Wenn-Dann-Regeln.
Ich lerne Muster in den Daten.
Deshalb kann ich auch Dinge erkennen,
die ich vorher noch nie gesehen habe.